**NMAM INSTITUTE OF TECHNOLOGY, NITTE**

Department of Information Science and Engineering

# OPERATING SYSTEMS

## REPORT

### AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Readers-Writers Problem and Dining Philosophers Problem

## SUBMITTED BY



| | |
|---|---|
| **Name** | Disha |
| **USN** | NNM24IS073 |
| **Course** | Operating Systems (OS) |
| **GitHub Repository** | https://github.com/DishaPoojary02/OS-task2 |



## SUBMITTED TO

Dr Jason Elroy Martis
Associate Professor
Dept Information Science and Engineering, Nitte

## 1.Problem Statement and Synchronization Objective

If several threads are acting on the same data at the same time, the final result may depend on the exact time each thread gets access to the CPU. This is called a race condition . The results are incorrect , and are difficult to reproduce . Operating systems use synchronization tools to handle this. This assignment is about solving two classic synchronization problems, each one two times, with two different tools: semaphores and monitors.

I built are four programs which are:

1. Dining Philosophers problem with four philosophers using a monitor
2. Dining Philosophers problem with four philosophers using semaphores
3. Readers-Writers problem using a monitor
4. Readers-Writers problem using semaphores

The objective is to show, in running code, how these tools give mutual exclusion, how deadlock is avoided in the philosophers problem, and where starvation could still sneak in. A second requirement is visual: every program has to write down what happened while it ran and then produce an HTML page that replays those events in a browser. The page must come from the real run of the program and must not be a hand drawn animation.

I chose Python for all four programs because its standard threading module already contains semaphores, locks and condition variables, so no extra installation is needed. Semaphores go back to Dijkstra's work on cooperating processes, and the monitor idea was described by Hoare

## 2. Readers-Writers: Semaphore Solution

The common resource is an integer variable named shared_data. The readers check its value, while the writers increment it by one. Critical sections are those portions of threads' code which access the common variable. The property of the algorithm is as follows: any number of readers may be in the critical section simultaneously, while the writer must be there alone.

### Semaphores used

| Name | Initial value | Job |
|---|---|---|
| mutex | 1 | Protects the counter read_count so two readers cannot update it at the same moment |
| resource | 1 | Gives access to the shared data: held by one writer, or by the group of readers as a whole |

There is an ordinary mutex log_lock as well, however, it protects only the array of logged events and is not used in the algorithm proper.

Initially, the reader changes its state to WAITING. Next, it acquires mutex, increments read_count and, if it is the first reader, acquires resource as well. Then it releases mutex and reads. On exiting, it acquires mutex once more, decrements read_count and, if it is the last reader, releases resource.

```
mutex.acquire()
read_count += 1
if read_count == 1:
    resource.acquire()   # first reader locks out the writers
mutex.release()
```

The key thing is that the resource.acquire() call is done by the very first reader, and all subsequent readers simply increment the counter and enter without blocking any other reader. Finally, the last reader calls the matching release().

## 2. Readers-Writers: Semaphore Solution

A writer is easier: it waits, calls resource.acquire(), adds one to shared_data, waits a bit of time randomly and then calls resource.release(). In case there are some readers in, this acquire will be blocked because the first reader has locked resources. The acquire can succeed only after all the readers have exited.

**Why it works**

There are two resources which are being protected here. mutex guarantees that the read_count will be incremented only by one reader at a time; otherwise two readers may see a value 0, add 1 to it, and have a count 1 instead of 2. In this case the last reader would release resource earlier than it should, because another reader is still inside the critical section. The resource guarantees that no writer will ever overlap with a reader or another writer.

**Starvation in this version**

This is a readers-preference solution. If there is at least one reader always inside, read_count is never decreased to zero and resources are never released, and so a writer may wait indefinitely. In my runs the threads make just a couple of attempts to get the lock, so the writers get a chance, but it is not guaranteed by the

## 3.Readers-Writers: Monitor Solution

**The monitor class**

Since there is no built-in monitor keyword, I implemented the behavior of such a construct in the form of a class called ReadWriteMonitor. It has one lock and two condition variables: can_read and can_write, which are based on this lock. Also, it has the following state, which is needed for decision making:

| Variable | Meaning |
|---|---|
| active_readers | How many readers are reading right now |
| writing | True while a writer is inside |
| waiting_writers | How many writers are waiting to enter |

Every method of the class acquires the lock first, which guarantees that only one thread will change these three variables at the same time. This is what is meant by mutual exclusion when using monitors.

**The four methods are:**

start_read(): waits until a writer is active or some writers are waiting, then increments the active_readers variable.

end_read(): decrements the active_readers variable, and if this value is now equal to zero, wakes up one waiting writer.

start_write(): increments the waiting_writers variable, waits until there are either any active readers or writers, then decrements the waiting_writers and sets the writing variable to true.

end_write(): sets the writing variable to false. If there are some waiting writers, wakes up one of them; otherwise, wakes up all waiting readers with notify_all().

When the thread is awakened, it does not immediately acquire the lock and other threads could change the state since then. Therefore, it is needed to check the condition again after the thread is awakened. To do this, the while loop should be used in every wait.

**Writer preference**

Because new readers wait whenever waiting_writers >0 zero, a stream of arriving readers cannot keep a writer out. The price is the opposite problem: if writers kept arriving all the time, readers could be delayed for long. This is a design choice, and I describe both sides in the starvation section.

## 4.Dining Philosophers (4 Philosophers): Semaphore Solution

There are four philosophers, numbered from 0 to 3, who sit at a table with four forks. Philosopher i uses fork i as the left fork and fork (i+1)%4 as the right fork. Thus each fork belongs to two neighbouring philosophers. A philosopher goes through the same cycle: thinking, getting hungry, acquiring the forks, eating, releasing the forks. The shared resources are the forks, while the critical section is the step of eating – in other words, when a philosopher gets both forks.

**Semaphores used**

| Name | Initial value | Job |
|---|---|---|
| forks[0..3] | 1 each | One semaphore per fork, so only one philosopher can hold a fork |
| room | 3 | Allows at most three philosophers to try for forks at the same time |

**The deadlock problem and the solution**

If each philosopher acquires the left fork first and then requests the right one, poor timing may cause all of them to acquire the left fork simultaneously. Then they wait for forks that belong to their neighbours and none of them will ever resume. This is a circular wait, one of the necessary conditions of a deadlock situation.

This can be avoided by making a philosopher first pass the room semaphore, which has been initialized to 3. Up to three philosophers can be inside the interval between acquiring the first fork and finishing his meal. Having four forks and up to three rivals, at least one of the philosophers is guaranteed to get both of his forks, eat and release them, thus breaking the deadlock circle. Room semaphore is released when the philosopher releases his forks.

**Ownership of the forks for visualization**

I maintain an array called fork_owner, which has an element for each fork. It equals -1 if the fork is not acquired, or the index of the philosopher who acquired it otherwise. I update it as soon as the philosopher acquires the fork and as soon as he releases it. The logged event records a copy of this array, and it is what makes it possible for the HTML page to know who owns what fork at each step.

## 4.Dining Philosophers (4 Philosophers): Semaphore Solution

### Starvation

The room semaphore avoids deadlock but does not guarantee fairness. The documentation for Python says nothing about order in which waiting threads are served, so I made no assumptions about it. In theory a philosopher might loose the race for a fork repeatedly. It was not the case in my experiments with three meals per philosopher, but the danger exists.

## 5. Dining Philosophers (4 Philosophers): Monitor Solution

### The monitor class

TableMonitor class includes one lock object, state, a list holding the current state of each philosopher, and one condition variable per philosopher, bound to the same lock. Instead of tracking forks as distinct objects, the monitor tracks eating philosophers because a philosopher can start eating only when neither of his neighbours is eating.

### pick_up and put_down

In pick_up(i) a philosopher marks himself as hungry and waits on his own condition variable until his left or his right neighbour finishes eating. When both become free, he notes fork picking up, marks himself as eating, and changes fork owners of his two forks. All this is done while the lock is held.

In put_down(i) he notes that he is releasing the forks, clears his two forks, returns to thinking and notifies both neighbours.

### How there is no deadlock

There is never only one fork taken at a time. The check and the pick up operation are done together while the lock is held. Thus, a philosopher never takes one fork and waits for another. The hold and wait condition is removed, hence there is no deadlock.

### Starvation

Starvation may still be an issue. For a table of four philosophers, there are no neighbours between 0 and 2 and between 1 and 3, so they can always eat simultaneously. If 0 and 2 keep alternately eating when they are free, philosophers 1 and 3 will always be blocked when they check. With random delays and limited number of rounds, it will be rare, but still possible as there is no mechanism that will prevent it. One possible way to resolve this problem is to give every waiting philosopher a ticket, see the improvements list.

## 6. How Events Are Recorded and Turned into HTML

**Recording**

In both algorithms there is a function log() that is called when a thread changes state. In Readers-Writers it writes the time of the change, the name of the thread, the new state, the value of shared_data variable and a comment. In Dining Philosophers it writes the time of the change, the number of the philosopher, the new state, the comment, and the fork_owner array.

Time is written in seconds from start of the program, using time.time(). Since all threads write to the same list, log() acquires a special lock log_lock before appending to the list. It guarantees that no two threads can write into the list at the same time.

**States are the following:**

Readers-Writers: thinking, waiting, reading, writing, done

Dining Philosophers: thinking, hungry, acquiring, eating, releasing

Calls of log() are placed immediately before and/or after the corresponding synchronization operation: the waiting event is logged right before the acquire, and the reading/writing/eating events – right after entering the critical section. Done/releasing events are logged right before giving up the lock, therefore the order of critical sections in the log corresponds to their actual order.

**Generating the page**

After all threads have completed (the main thread waits for them using join()), the function make_html() writes the list of events as JSON text using json.dumps() and embeds it into a string of HTML template in place of a placeholder. The resulting HTML page contains the full trace of that particular run.

**Replay in the browser**

The JavaScript in the page maintains the current state of every thread. The function step() takes the next event, updates the state of the threads, redraws the visualization and appends a new line to the events log box. The timer calls step() periodically; the Start/Pause/Reset buttons and the speed box control the timer.

In case of Readers-Writers, the page displays one colored box per each reader and writer, the panel with the shared resource, and the timeline. The resource panel calculates the displayed text from the states: free, open for some number of readers or locked for writing. In case of Dining Philosophers, the page visualizes four philosophers sitting around the table with four forks. Free forks are drawn as green objects on the table, and held forks as red ones that are moved to the philosopher who holds it. Also, the side panel shows the owner of each fork.

## 7. Execution Traces

**Readers-Writers with semaphores**

[0.001s] Reader 1 -> thinking

[0.001s] Reader 2 -> thinking

[0.002s] Reader 3 -> thinking

[0.002s] Writer 1 -> thinking

[0.002s] Writer 2 -> thinking

[0.206s] Writer 1 -> waiting wants to write

[0.206s] Writer 1 -> writing wrote value 1

[0.215s] Writer 2 -> waiting wants to write

[0.256s] Reader 1 -> waiting wants to read

[0.507s] Reader 2 -> waiting wants to read

[0.508s] Reader 3 -> waiting wants to read

[0.588s] Writer 1 -> done finished writing

[0.589s] Writer 1 -> thinking

**Readers-Writers with a monitor**

[0.002s] Reader 1 -> thinking

[0.003s] Reader 2 -> thinking

[0.003s] Reader 3 -> thinking

[0.003s] Writer 1 -> thinking

[0.004s] Writer 2 -> thinking

[0.132s] Reader 1 -> waiting wants to read

[0.132s] Reader 1 -> reading read value 0

[0.161s] Writer 2 -> waiting wants to write

## 7. Execution Traces

[0.184s] Writer 1 -> waiting wants to write

[0.304s] Reader 3 -> waiting wants to read

[0.468s] Reader 2 -> waiting wants to read

[0.494s] Reader 1 -> done finished reading

[0.494s] Reader 1 -> thinking

**Dining Philosophers with semaphores**

[0.001s] P0 -> thinking

[0.003s] P1 -> thinking

[0.003s] P2 -> thinking

[0.003s] P3 -> thinking

[0.183s] P1 -> hungry wants to eat

[0.183s] P1 -> acquiring picking up forks

[0.183s] P1 -> acquiring has left fork 1

[0.183s] P1 -> eating has forks 1 and 2

[0.189s] P2 -> hungry wants to eat

[0.189s] P2 -> acquiring picking up forks

[0.258s] P3 -> hungry wants to eat

[0.258s] P3 -> acquiring picking up forks

[0.258s] P3 -> acquiring has left fork 3

## 7. Execution Traces

**Dining Philosophers with a monitor**

[0.002s] P0 -> thinking

[0.004s] P1 -> thinking

[0.004s] P2 -> thinking

[0.005s] P3 -> thinking

[0.122s] P2 -> hungry wants to eat

[0.122s] P2 -> acquiring picking up forks 2 and 3

[0.122s] P2 -> eating has both forks

[0.235s] P1 -> hungry wants to eat

[0.314s] P3 -> hungry wants to eat

[0.55s] P0 -> hungry wants to eat

[0.55s] P0 -> acquiring picking up forks 0 and 1

[0.55s] P0 -> eating has both forks

[0.711s] P2 -> releasing putting forks down

[0.711s] P2 -> thinking

## 8. Race Conditions, Mutual Exclusion, Deadlock and Starvation

### Race conditions

Race condition arises when the result depends on the sequence of thread execution. Python has a global interpreter lock, yet it doesn't guarantee safety of these programs. Read_count += 1 operation consists of read, increment and write actions, and the interpreter may switch threads in the middle of them.

In my programs there were four unprotected accesses:

read_count which is protected by mutex in the semaphore version and by monitor lock in the monitor version

shared_data which is modified only by the writer with exclusive access

list of events which is protected by log_lock

fork_owner list and philosopher states in the philosophers programs which are modified only by the thread holding appropriate fork or monitor lock

### Mutual exclusion

In case of Readers-Writers mutual exclusion implies that writer is never inside the program together with any readers and writers. In semaphore version of the solution mutual exclusion is provided by resource, while in the monitor version it is guaranteed by conditions on writing and active_readers under the lock. In Dining Philosophers mutual exclusion means that a fork is owned by at most one philosopher, provided either by fork semaphores in the first version or by neighbor check in the second.

### Deadlock

For deadlock we need all four conditions: mutual exclusion, hold and wait, no preemption and circular wait [2]. Fork can not be withdrawn from a philosopher, and a fork is owned by at most one philosopher at a time, hence these conditions are always satisfied in this problem. Solutions break one of the other two:

semaphore version – room semaphore ensures that there are at most three competing philosophers in the room, so the circular wait can't happen;

monitor version – both forks are acquired simultaneously under the lock. Hold and wait condition is broken.

The Readers-Writers solutions can not go into deadlock for the same reason.

## 8. Race Conditions, Mutual Exclusion, Deadlock and Starvation

### Starvation

Starvation indicates that a thread has been waiting for too long while the other threads keep being served.

| Program | Who can starve | Why |
|---|---|---|
| Readers-Writers, semaphore | Writers | Readers keep the resource locked as long as one of them is always inside |
| Readers-Writers, monitor | Readers | New readers wait whenever a writer is waiting, so a steady stream of writers could delay them |
| Philosophers, semaphore | Any philosopher | No guaranteed order among waiting threads |
| Philosophers, monitor | Philosophers 1 and 3 (or 0 and 2) | Two non-neighbours can keep taking turns and block the others |

These did not appear as a permanent block in my tests, since every thread performs a finite number of tries. However, it does not prove that they can never occur.

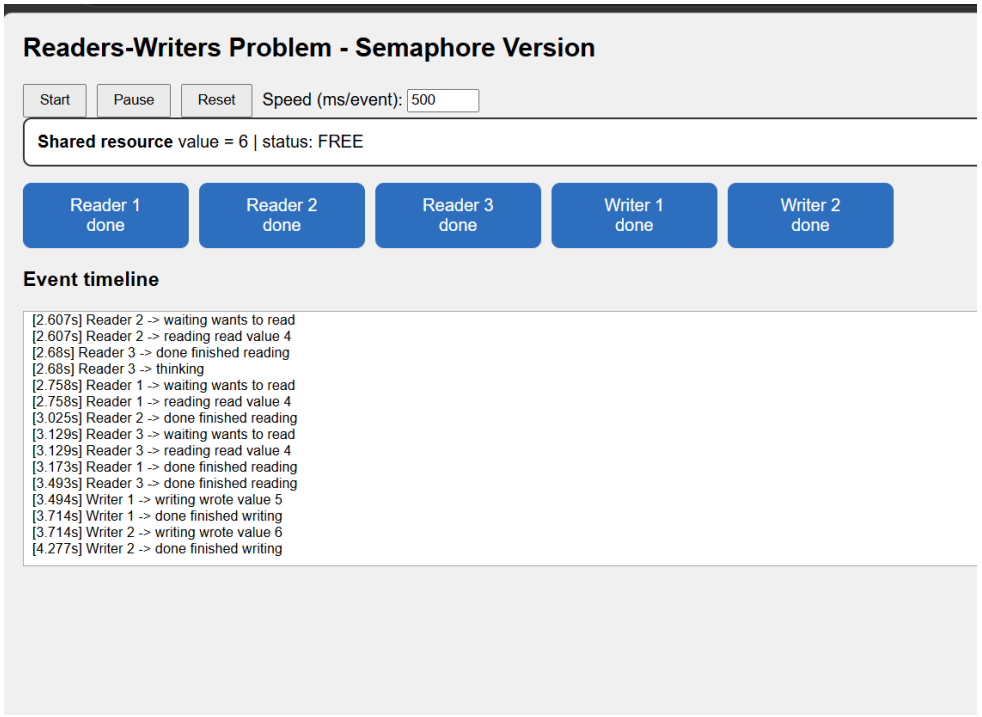
## 9. Semaphores Compared with Monitor

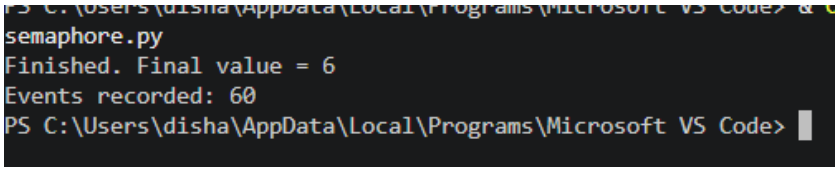
| Point | Semaphore approach | Monitor approach |
|---|---|---|
| Where the logic lives | Spread over the reader and writer code, with acquire and release calls around the critical section | Collected inside one class with entry and exit methods |
| How threads wait | They block on acquire() | They wait on a condition variable inside a while loop |
| Typical mistake | Forgetting a release, or releasing in the wrong order, which can block everyone | Forgetting to notify, or using if instead of while |
| Information about the state | Hidden in the semaphore value and in helper counters such as read_count | Visible as ordinary variables such as writing and waiting_writers |
| Flexibility | Very flexible, low-level building block | Higher-level, but fewer ways to go wrong |
| Policy changes | Harder, as it often needs extra semaphores | Easy, by changing the condition in one place |

Writing both versions made me appreciate the difference between them. The semaphore version was shorter, but it was necessary to keep track of the release order of semaphores, and the first-reader and last-reader technique is prone to mistakes. The monitor version was more verbose, but every condition was explicitly stated in a single line and was easier to modify, for instance when I changed the preference of readers over writers. Both of them allow solving the same issues, but the monitor solution relies more on the structure itself and less on the programmer's self-discipline.

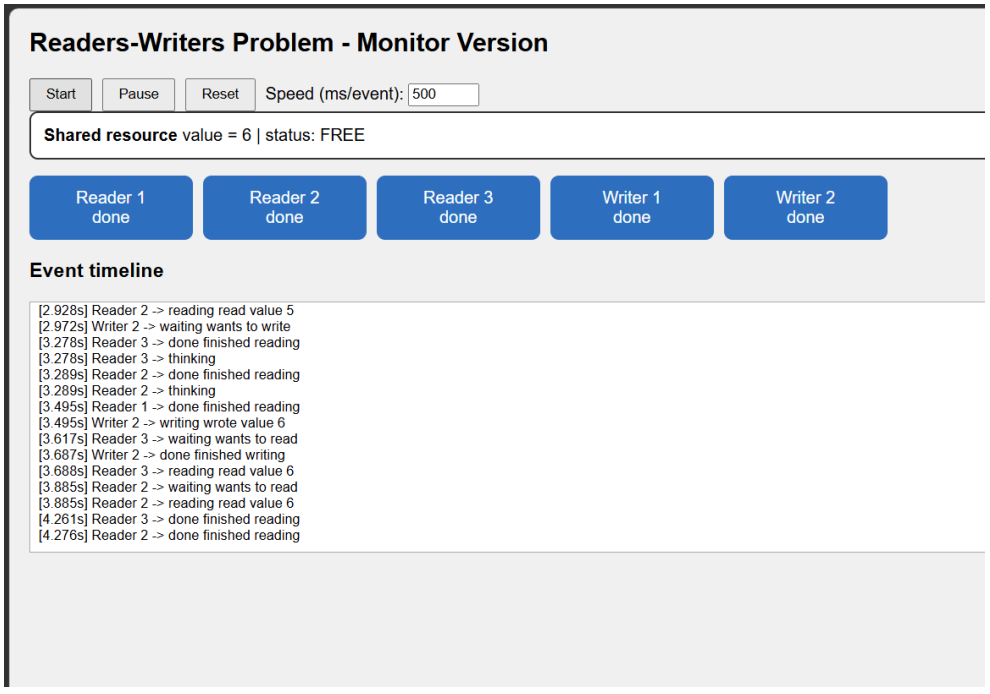
# 10. Screenshots

## Readers-Writers Using Semaphore




## Readers-Writers Using Monitors

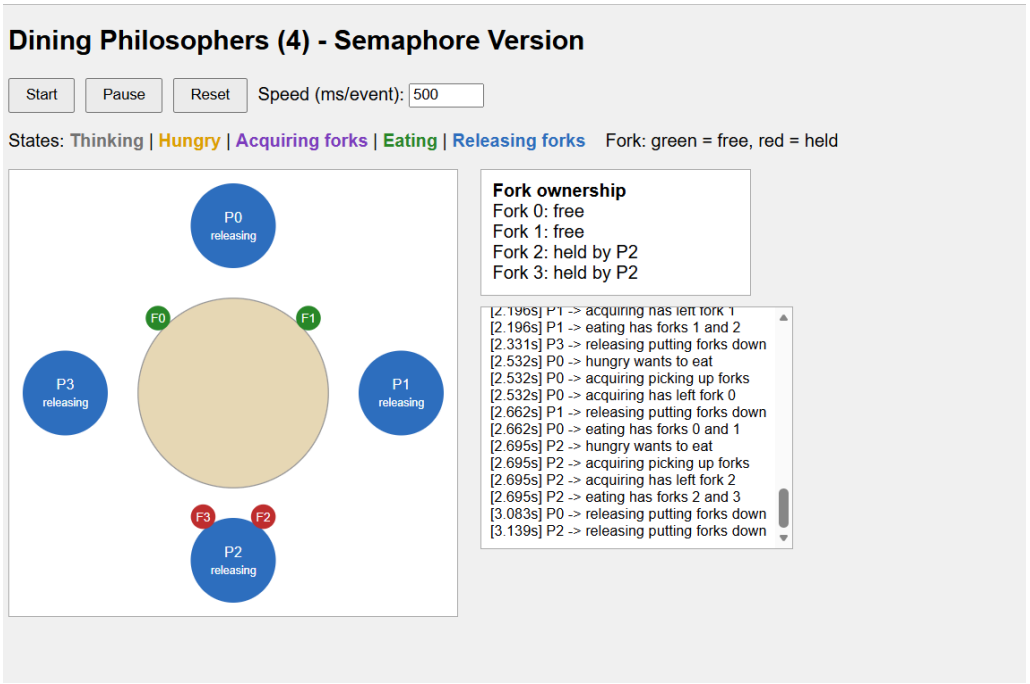


# 10. Screenshots

```
PS C:\Users\disha\AppData\Local\Programs\Microsoft VS Code> & C:\Us
nitor.py
Finished. Final value = 6
Events recorded: 60
PS C:\Users\disha\AppData\Local\Programs\Microsoft VS Code>
```

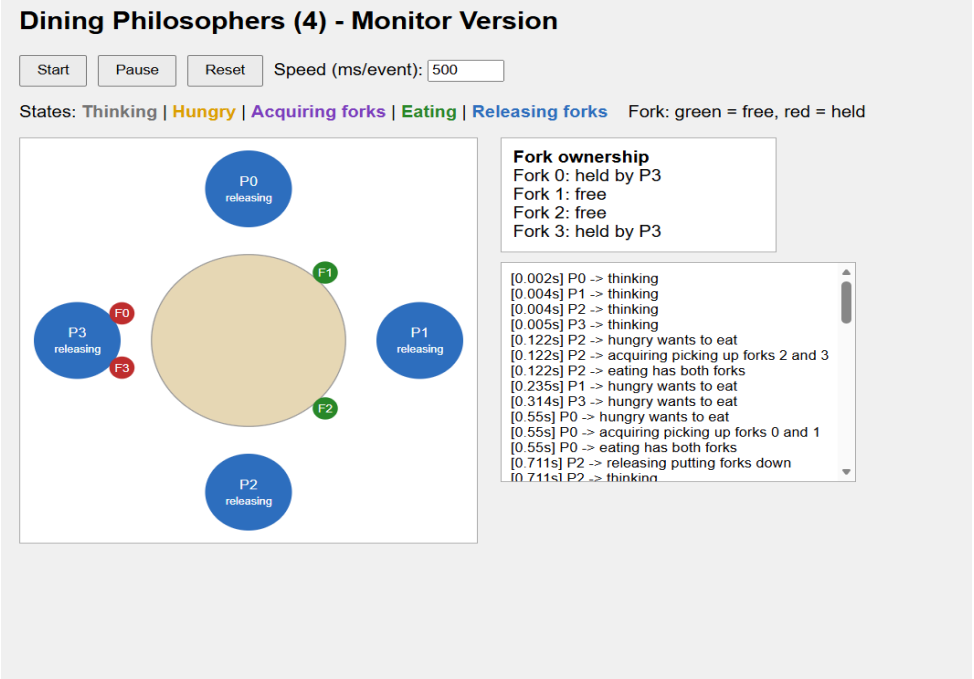
## Dining Philosophers Using Semaphore

```
ation-Assignment.zip.364/dining_philosophers/semaphore/dp_semaphore.py
Finished. Every philosopher ate 3 times, no deadlock.
Events recorded: 72
```



# 10. Screenshots

## Dining Philosophers Using Monitor




```
ation-Assignment.zip.bfa/dining_philosophers/monitor/dp_monitor.py
Finished. Every philosopher ate 3 times, no deadlock.
Events recorded: 60
```

## 11. AI Usage Summary

The AI assistant used: Claude

I used the AI assistant to comprehend the requirements of the task, to create the folder structure, to write the initial version of all four programs and their HTML representations, and to receive explanations of the synchronization techniques. The prompts I have used are provided in ai_prompts/prompts.md.

I executed every program on my own computer, and opened every generated page. I also read through the code of each program in order to comprehend the use of every semaphore, lock and condition variable and describe it.

The limitation in the AI-generated output: All generated programs display the behavior of only a few rounds, so the phenomenon of starvation cannot be observed even in those programs where it may appear. A successful run, therefore, is weak evidence of fairness of the algorithm, and the starvation discussion in this report is based on the reasoning about algorithms, rather than on observation.

## 12. References


[1] Deitel, Harvey M., and Paul J. Deitel. *Operating systems*. Pearson Education India, 2004.

[2] Eager, Bob, and Andrew Lister. *Fundamentals of operating systems*. Bloomsbury Publishing, 2016.

[3] Naghibzadeh, Mahmoud. *Operating System: Concepts and Techniques*. IUniverse, 2005.

[4] Chun, Wesley. *Core python programming*. Vol. 1. Prentice Hall Professional, 2001.

[5] Adams, Joel C., Elizabeth R. Koning, and Christiaan D. Hazlett. "Visualizing classic synchronization problems: Dining philosophers, producers-consumers, and readers-writers." *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*. 2019.

[6] Heymans, F., David Lorge Parnas, and Pierre Courtois. "Concurrent control with “readers” and “writers”." *Communications of the ACM* 14.10 (1971): 667-668.

[7] Chandy, K. Mani, and Jayadev Misra. "The drinking philosophers problem." *ACM Transactions on Programming Languages and Systems (TOPLAS)* 6.4 (1984): 632-646.

[8] Adams, Joel C., Elizabeth R. Koning, and Christiaan D. Hazlett. "Visualizing classic synchronization problems: Dining philosophers, producers-consumers, and readers-writers." *Proceedings of the 50th ACM Technical Symposium on Computer Science Education*. 2019.

[9] Feldman, Michael B. "The portable dining philosophers: a movable feast of concurrency and software engineering." *ACM SIGCSE Bulletin* 24.1 (1992): 276-280.

[10] Iglesias, Manuel José Fernández. "Brief Introduction to Operating Systems."

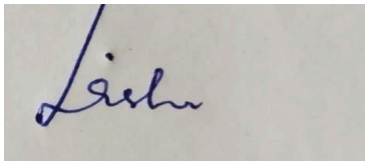
## 13. Academic Integrity Declaration

I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own.

**Name:** Disha

**USN:** NNM24IS073

**Signature:**



**Date: 26/10/26**